



**SCHOOL OF
COMPUTING**

DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK

WEEK - 8

NAME : SRINIVAS R

ROLL NUMBER : CH.SC.U4CSE24146

CLASS : CSE-B

Question 1: Write a C program to implement Huffman coding that takes a string as input from the user, calculates the frequency of each character, and generates Huffman codes for all unique characters present in the string. Display the character, its frequency, and the corresponding Huffman code in a tabular format.

Consider the following input string representing a dataset label:

"DATAANALYTICSANDINTELLENCELABORATORY"

CODE:

```
//ch.sc.u4cse24146
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define ASCII_SIZE 256
#define MAX_HEIGHT 100

/* Node structure for Huffman Tree */
typedef struct Node {
    char ch;
    int frequency;
    struct Node *leftChild;
    struct Node *rightChild;
} Node;

/* Min Heap structure */
typedef struct Heap {
    int currentSize;
    int maxSize;
    Node **nodes;
} Heap;

/* Create new tree node */
Node* createNode(char ch, int frequency) {
    Node* temp = (Node*)malloc(sizeof(Node));
    temp->ch = ch;
    temp->frequency = frequency;
    temp->leftChild = NULL;
    temp->rightChild = NULL;
    return temp;
}

/* Create heap */
Heap* initializeHeap(int capacity) {
    Heap* heap = (Heap*)malloc(sizeof(Heap));
    heap->currentSize = 0;
    heap->maxSize = capacity;
    heap->nodes = (Node**)malloc(capacity * sizeof(Node*));
    return heap;
}

/* Swap two heap nodes */
void swapNodes(Node** a, Node** b) {
    Node* temp = *a;
    *a = *b;
    *b = temp;
}

/* Heapify */
void heapify(Heap* heap, int index) {
    int smallest = index;
```

```

int left = 2 * index + 1;
int right = 2 * index + 2;

if (left < heap->currentSize &&
    heap->nodes[left]->frequency <
    heap->nodes[smallest]->frequency)
    smallest = left;

if (right < heap->currentSize &&
    heap->nodes[right]->frequency <
    heap->nodes[smallest]->frequency)
    smallest = right;

if (smallest != index) {
    swapNodes(&heap->nodes[smallest],
              &heap->nodes[index]);
    heapify(heap, smallest);
}
}

/* Remove minimum node */
Node* removeMin(Heap* heap) {
    Node* temp = heap->nodes[0];
    heap->nodes[0] = heap->nodes[heap->currentSize - 1];
    heap->currentSize--;
    heapify(heap, 0);
    return temp;
}

/* Insert node into heap */
void addToHeap(Heap* heap, Node* node) {
    heap->currentSize++;
    int i = heap->currentSize - 1;

    while (i > 0 &&
           node->frequency <
           heap->nodes[(i - 1) / 2]->frequency) {
        heap->nodes[i] =
            heap->nodes[(i - 1) / 2];
        i = (i - 1) / 2;
    }

    heap->nodes[i] = node;
}

/* Build initial heap */
void buildHeap(Heap* heap) {
    for (int i = (heap->currentSize - 2) / 2; i >= 0; i--)
        heapify(heap, i);
}

/* Check if leaf */
int isLeafNode(Node* root) {
    return (!root->leftChild && !root->rightChild);
}

/* Print Huffman Codes */
void displayCodes(Node* root, int code[], int level) {

    if (root->leftChild) {
        code[level] = 0;
        displayCodes(root->leftChild, code, level + 1);
    }
}

```

```

    if (root->rightChild) {
        code[level] = 1;
        displayCodes(root->rightChild, code, level + 1);
    }

    if (isLeafNode(root)) {
        printf("%c\t%d\t", root->ch, root->frequency);
        for (int i = 0; i < level; i++)
            printf("%d", code[i]);
        printf("\n");
    }
}

/* Construct Huffman Tree */
void generateHuffman(char characters[],
                    int frequencies[],
                    int count) {

    Heap* heap = initializeHeap(count);

    for (int i = 0; i < count; i++)
        heap->nodes[i] =
            createNode(characters[i], frequencies[i]);

    heap->currentSize = count;
    buildHeap(heap);

    while (heap->currentSize > 1) {
        Node* left = removeMin(heap);
        Node* right = removeMin(heap);

        Node* merged =
            createNode('#',
                        left->frequency + right->frequency);

        merged->leftChild = left;
        merged->rightChild = right;

        addToHeap(heap, merged);
    }

    int code[MAX_HEIGHT];
    printf("\nCharacter  Frequency  Code\n");
    printf("-----\n");

    displayCodes(removeMin(heap), code, 0);
}

int main() {

    char input[1000];
    int frequency[ASCII_SIZE] = {0};

    printf("Enter dataset label: ");
    scanf("%s", input);

    /* Count frequency */
    for (int i = 0; input[i] != '\0'; i++)
        frequency[(unsigned char)input[i]]++;

    char chars[ASCII_SIZE];
    int freqs[ASCII_SIZE];
    int uniqueCount = 0;

```

```

for (int i = 0; i < ASCII_SIZE; i++) {
    if (frequency[i] > 0) {
        chars[uniqueCount] = (char)i;
        freqs[uniqueCount] = frequency[i];
        uniqueCount++;
    }
}

generateHuffman(chars, freqs, uniqueCount);

return 0;
}

```

OUTPUT:

```

D:\WEEK-8\week_8.exe
Enter dataset label: DATAANALYTICSANDINTELLIGENCELABORATORY

Character  Frequency  Code
-----
N          4        000
L          4        001
O          2        0100
R          2        0101
C          2        0110
B          1        01110
G          1        01111
T          4        100
Y          2        1010
S          1        10110
D          2        10111
E          3        1100
I          3        1101
A          7        111

-----
Process exited after 54.87 seconds with return value 0
Press any key to continue . . .

```

Space Complexity: $O(N+M)$: The space complexity is $O(n + m)$. The input string requires $O(n)$ space, while the frequency array, min-heap, and Huffman tree nodes together require $O(m)$ space. Because the number of possible ASCII characters is limited to 256, m is bounded by a constant in practical situations, so the space complexity effectively behaves like $O(n)$.

Time Complexity: $O(N+ M \log M)$: $O(n + m \log m)$ where N is the input string length and M is the number of unique characters - the frequency counting takes $O(N)$ while building the Huffman tree with m nodes takes $O(M \log M)$ due to repeated extractMin and insert operations in the min-heap.

